
Search Engine Project

Stage 1: Building the Data Layer

Java implementation — technical report

Course	Big Data · Grado en Ciencia e Ingeniería de Datos · Universidad de Las Palmas de Gran Canaria
Academic year	2026/2027
Group name	BigData-Ulpgc
Members	Daniel Rodríguez Alonso — Java implementation, author of this report Daniel Perdomo Medina — Python implementation Carlos Falcón Brito — C++ implementation
Repository	https://github.com/BigData-Ulpgc/stage_1 , branch java, folder java/stage1/
Status	Complete pipeline, offline mode with the sample dataset, 12 benchmark experiments, 357 tests passing
Date	4 October 2026

Quick check for instructors. From java/stage1/, with JDK 17+ and Maven, and no network: mvn package, then `java -cp "$CP" es.ulpgc.bigdata.Main pipeline 30 --offline` and `java -cp "$CP" es.ulpgc.bigdata.Main search whale island`. Section 6.3 gives the exact commands and the expected output.

Abstract

This report describes the Java implementation of Stage 1 of the group’s search engine over Project Gutenberg books. Stage 1 builds the *data layer*: a pipeline that downloads books, splits each one into header and body, stores them in a datalake, extracts their metadata into a SQLite datamart and builds an inverted index that answers Boolean AND queries. A control layer records the progress so an interrupted run resumes without losing or repeating books.

The implementation offers three datalake layouts (**book**, **range**, **time**) and three persistent inverted-index structures (**monolithic** JSON, **hierarchical** folders, **mongo**), selectable through configuration without touching code. It follows the group’s common contract (**shared/SPEC.md**), so its outputs match the Python and C++ implementations. A 15-book sample dataset lets the whole pipeline run offline in about one second. The 12 benchmark experiments, run on the 200 real books of the dataset, show that the **monolithic** index is the fastest to build and query but is the only one whose update cost and memory grow with the index; that **hierarchical** wastes about 73 times its data size in file-system blocks; and that the **time** datalake pays for its arrival-order layout with lookups about 29 times slower than **book**.

Contents

1	Introduction and objectives	3
1.1	Goal of Stage 1	3
1.2	The group and the common contract	3
1.3	Objectives and scope of this report	4
2	System architecture	5
2.1	The life of a book	5
2.2	Layers and dependencies	6
2.3	Design principles applied throughout	6
2.4	Project organisation	6
2.4.1	Folder tree	7
2.4.2	Size of the code	7
2.4.3	Where each part of the state lives	7
3	Design decisions	8
3.1	Structures selected for the final implementation	8
3.2	Implementation decisions	9
4	Implementation: packages and classes	9
4.1	Package <code>model</code> : the data that travels	10
4.2	Package <code>crawler</code> : fetching and splitting	10
4.3	Package <code>datalake</code> : storing the books	11
4.4	Package <code>datamart.metadata</code> : metadata in SQLite	12
4.5	Package <code>datamart.index</code> : the inverted index	13

4.6	Package <code>query</code> : the search	15
4.7	Package <code>control</code> : progress and recovery	16
4.8	Package <code>config</code> and assembly	16
4.9	Package <code>benchmark</code>	17
5	Testing strategy	18
5.1	The end-to-end test	18
5.2	The sample dataset test	19
6	Setup and execution	19
6.1	Requirements	19
6.2	Build and test	19
6.3	Quick test without network: the sample dataset	19
6.4	Commands	20
6.5	Configuration: changing structures without touching code	21
6.6	Running the benchmarks	21
7	Benchmarks and results	22
7.1	Methodology	22
7.2	Data: real and synthetic books	22
7.3	Datalake benchmark	23
7.4	Metadata benchmark	24
7.5	Inverted-index benchmark	25
7.6	Real versus synthetic data	28
7.7	Summary: which structure to choose	29
8	Conclusions and future improvements	30
8.1	Conclusions	30
8.2	Limitations and future improvements	30

1. Introduction and objectives

1.1 Goal of Stage 1

The final goal of the course project is a search engine over books from Project Gutenberg. Stage 1 does not yet aim at ranking or a user interface. It builds the data layer underneath: books must be downloaded, stored, described and indexed *correctly*, in a way that *recovers from failures*, and *measurably*, so that several ways of organising the data can be compared.

Layer	What it does	Java package
Ingestion	Downloads the text from Gutenberg and splits it into header (title, author...) and body (the book itself); the licence footer is discarded.	<code>crawler</code>
Datalake	Stores header and body as files, in one of three layouts: book , range or time .	<code>datalake</code>
Datamarts	Processed data ready to query: metadata in SQLite, and the inverted index (term → books) in three variants.	<code>datamart.*</code>
Query	AND search: the books that contain every term of the query.	<code>query</code>
Control	Files that remember which books are downloaded and indexed, so work is never repeated or lost.	<code>control</code>
Configuration	One place for paths, timeouts and active structures; factories that map names to implementations.	<code>config</code>
Benchmarks	Measure and compare the structures with a common methodology, writing CSV results.	<code>benchmark</code>

Table 1: The layers of the system and where each lives in the Java code.

1.2 The group and the common contract

The group implements the same data layer three times: in Java (this report), Python and C++. So that the three can be compared, the group agreed on a common contract, `shared/SPEC.md`, which fixes everything that must not differ between languages. Many design decisions in the Java code follow directly from it.

SPEC section	Rule
1. Dataset	<code>book_ids.txt</code> (ids to download, in order), <code>queries.txt</code> (benchmark queries), <code>stopwords.txt</code> . Empty lines and lines starting with # are ignored.
2. Split	Markers *** START OF THE/THIS PROJECT GUTENBERG EBOOK and *** END OF If either is missing, the book is discarded. Header = text before START (trimmed); body = from the end of the START line to END (trimmed); <code>\r\n</code> → <code>\n</code> .
3. Datalake	<code>time: YYYYMMDD/HH/<ID>.body.txt</code> ; <code>book: <ID>/body.txt</code> ; <code>range: <START>-<END>/<ID>.body.txt</code> , with <code>START = ⌊ID/1000⌋ · 1000</code> in 5 digits.
4. Metadata	<code>title</code> , <code>author</code> , <code>release_date</code> , <code>language</code> by multi-line regex, first match, first line only, trimmed; missing → NULL. SQLite table <code>books</code> with indexes on <code>author</code> and <code>title</code> .
5. Tokenizer	Character by character: A–Z → lowercase; a–z and 0–9 form tokens; anything else separates (accents and apostrophes included). Tokens shorter than 2 and stopwords are dropped. Each book contributes its <i>set</i> of terms (no frequencies).
6. Index	monolithic : one JSON <code>{"term": [ids]}</code> ; hierarchical : <code>inverted_index/<LETTER>/<term>.txt</code> ; mongo : collection <code>inverted_index</code> with a unique index on <code>term</code> . Posting lists sorted, without duplicates.
7. Queries	The query goes through the same tokenizer; AND = intersection; ids sorted.
8. Control	<code>control/downloaded_books.txt</code> and <code>indexed_books.txt</code> ; an id is appended only after its step succeeded.
9–10. Benchmarks	CSV <code>language,experiment,structure,dataset_size,repetition,metric,value,unit</code> ; 2 warm-ups + 5 measured runs; no network inside a measurement; the 200 real books, size N = the N lowest ids; reference term and posting counts.

Table 2: Summary of the common contract, `shared/SPEC.md`.

1.3 Objectives and scope of this report

The objectives of Stage 1, as set by the assignment, are to build a datalake for the raw books and datamarts for structured, queryable data (metadata and inverted index); to coordinate them with a minimal control layer that never duplicates or loses work; and to benchmark alternative datalake and inverted-index structures, implemented in at least three programming languages over the same data.

This report covers the **Java** implementation of the group’s project. It explains its architecture, its design decisions and every class of its code, how to run and test it, and the results of its benchmarks. The Python and C++ implementations follow the same contract and are documented in their own folders of the repository.

The Java implementation delivers:

- The complete pipeline: crawler, datalake, metadata datamart, inverted-index datamart, AND query and control layer, behind a small command-line interface (**Main**).
- All three datalake layouts and all three persistent index structures of the SPEC, plus an in-memory index used as a reference in tests. The active ones are chosen in `config.properties` or with `-D` options.
- An **offline mode** that runs the same pipeline on the 15 books of `sample_dataset/`, so instructors can test everything without network.
- All 12 benchmark experiments of the SPEC, in its CSV format, with results committed in `benchmarks/results/real/` and `synthetic/`.
- 357 JUnit 5 tests at four levels (unit, contract, integration and end-to-end), including one

that checks the output against the sample dataset's reference values.

2. System architecture

Figure 1 follows one book through the system. Every storage box is an interface with several interchangeable implementations.

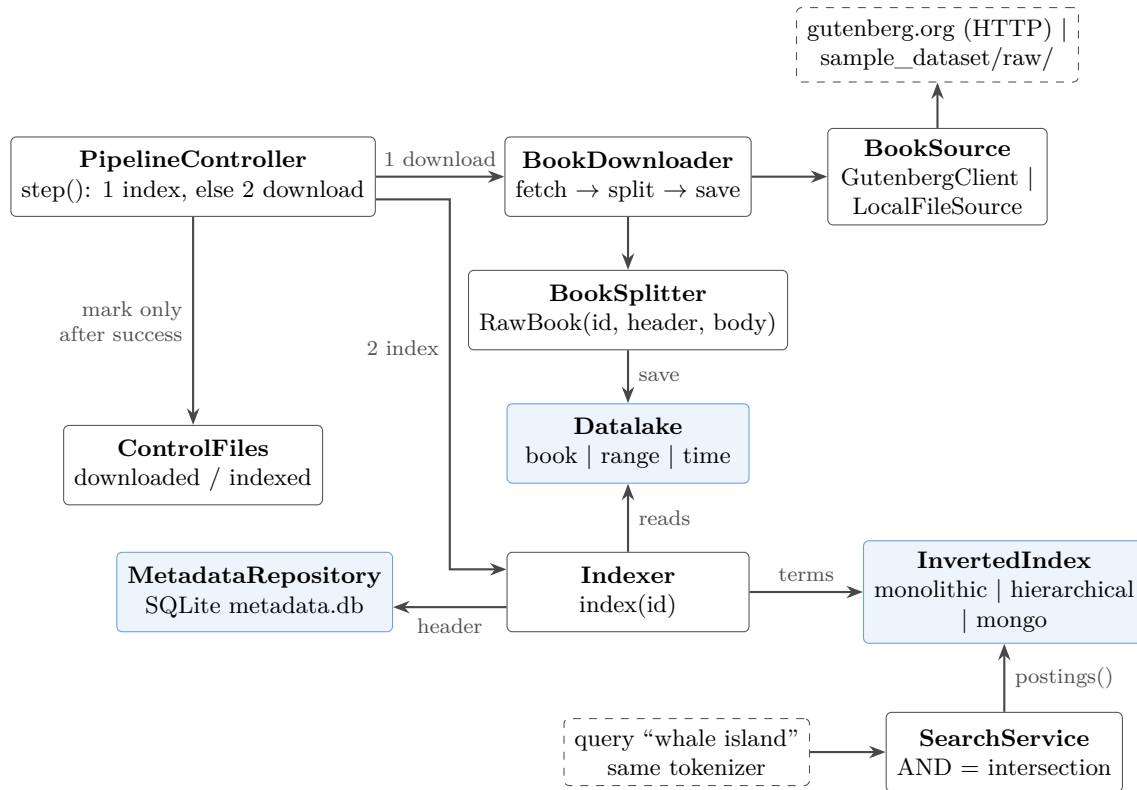


Figure 1: Main components and data flow. **SearchEngine** creates and connects every piece from an **AppConfig**, using **DatalakeFactory** and **InvertedIndexFactory**.

2.1 The life of a book

1. **PipelineController.step()** reads the control files. Nothing is waiting to be indexed, so it picks the first id of **book_ids.txt** that is not downloaded yet, for example 1342.
2. **BookDownloader.download(1342)**: **GutenbergClient.fetch** downloads the **.txt** over HTTP (or **LocalFileSource** reads it from the sample dataset) → **BookSplitter.split** finds the markers and returns a **RawBook(id, header, body)** → **Datalake.save** writes header and body atomically and returns a **BookLocation**.
3. Only if everything succeeded, **ControlFiles.markDownloaded(1342)** appends “1342” to **downloaded_books.txt**.
4. In the next step there is a downloaded book that is not indexed. **Indexer.index(1342)** reads header and body from the datalake and computes *everything* before writing anything (metadata with **MetadataParser**, terms with **Tokenizer**). Then it saves the metadata in SQLite, adds the book to the index and calls **flush()**.
5. Only then, **ControlFiles.markIndexed(1342)**.
6. A query “whale island” goes through **SearchService.search**: it is tokenized like the books,

the posting lists of **whale** and **island** are requested from the index, and they are intersected.

2.2 Layers and dependencies

Dependencies always point downwards: higher layers know lower ones, never the other way round. Business classes depend on interfaces (**Datalake**, **InvertedIndex**, **MetadataRepository**, **BookSource**); only the assembly layer knows which concrete class is used.

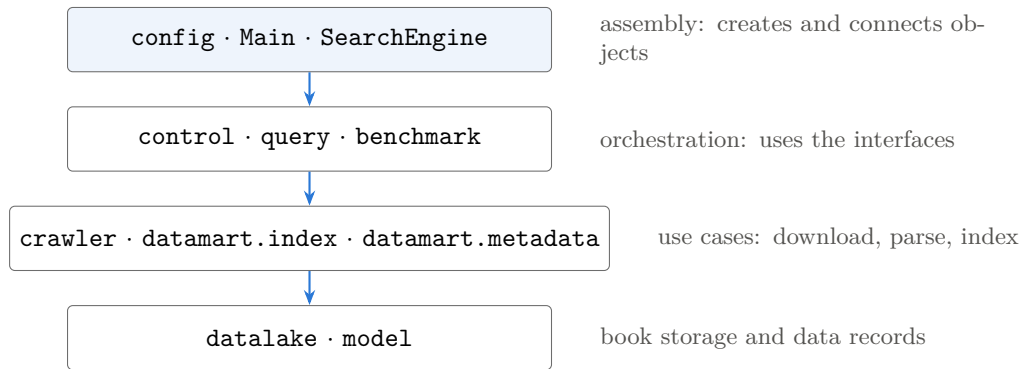


Figure 2: Layers. Each one only uses the layers below it.

2.3 Design principles applied throughout

Principle	Where	Why
Programming to interfaces	Datalake , InvertedIndex , MetadataRepository , BookSource	Changing a structure does not touch its users; tests use doubles (lambdas, in-memory index).
Constructor injection	Every class receives its collaborators in the constructor.	Nothing is created behind the scenes; any piece can be replaced in tests.
Factory	DatalakeFactory , InvertedIndexFactory	One configuration name → one implementation, in a single place.
Template method	AbstractFileDatalake : subclasses decide <i>where</i> ; the base decides <i>how</i> a book is written and what counts as saved.	The delicate logic (atomicity) exists only once.
Atomic writes	Datalake , JSON index, term files, CSV results: write .tmp , then rename.	A crash never leaves a half-written file under its final name.
Idempotence	save (upsert), addDocument (sets), mark*	Repeating an operation after a failure is safe: nothing is duplicated.
Mark after doing	PipelineController : mark* only after success.	The control layer never claims something that did not happen; at worst work is repeated.
Immutable records	RawBook , BookLocation , BookMetadata , StepResult , AppConfig	Data without behaviour, automatic equals, impossible to modify by accident.

Table 3: Design principles and where they appear in the code.

2.4 Project organisation

2.4.1 Folder tree

```

Stage_1/
|-- README.md           overview of the group project
|-- docker-compose.yml  MongoDB 8.2 server shared by the three implementations
|-- shared/             COMMON CONTRACT (Java, Python and C++)
|   |-- SPEC.md         rules that must not differ between languages
|   |-- book_ids.txt    the 200 books of the dataset, in order
|   |-- queries.txt     AND queries of the index_query benchmark
|   |-- stopwords.txt   English stopwords
|-- sample_dataset/     first 15 books, raw (input) and split (expected output)
'-- java/stage1/        <- the Java implementation (run everything from here)
    |-- pom.xml         Maven: Java 17, JUnit 5, sqlite-jdbc, Jackson, Mongo driver
    |-- config.properties default configuration (read by AppConfig)
    |-- README.md       user guide of the module
    |-- docs/           this report; report/ holds its LaTeX source and charts
    |-- src/main/java/es/ulpgc/bigdata/
    |   |-- Main.java   command line: pipeline | search | status | config
    |   |-- SearchEngine.java assembles the whole system from an AppConfig
    |   |-- model/       RawBook, BookLocation, BookMetadata (records)
    |   |-- crawler/     BookSource, GutenbergClient, LocalFileSource,
    |   |               BookSplitter, BookDownloader
    |   |-- datalake/     Datalake, AbstractFileDatalake, Book/Range/TimeBasedDatalake
    |   |-- datamart/metadata/ MetadataParser, MetadataRepository, SQLite
    |   |-- datamart/index/ Tokenizer, InvertedIndex + 4 implementations, Indexer
    |   |-- query/       SearchService (AND)
    |   |-- control/     ControlFiles, PipelineController, StepResult, BookIdList
    |   |-- config/      AppConfig, DatalakeFactory, InvertedIndexFactory
    |   |-- benchmark/   BenchmarkRunner, CSV output and the three benchmarks
    |-- src/test/java/... 357 tests (JUnit 5), same package structure
    |-- data/             (generated, ignored by git) datalake, datamarts, control
    '-- benchmarks/      results/{real,synthetic}/*.csv (committed), work/ (ignored)

```

2.4.2 Size of the code

Item	Amount
Production classes and interfaces	44 (about 4,800 lines with comments)
Test classes	35 (about 5,200 lines), 357 tests
External dependencies	sqlite-jdbc 3.53, jackson-databind 2.22, mongodb-driver-sync 5.13, JUnit 5

2.4.3 Where each part of the state lives

All paths are built by `AppConfig` from `data.dir` (`data/` by default, relative to `java/stage1`). No other class writes a data path; a test fails if one does.

State	File or database	Written by
Book text	data/datalake/<structure>/...	Datalake.save
Metadata	data/datamarts/metadata.db (table books)	SqliteMetadataRepository
Monolithic index	data/datamarts/ inverted_index.json	MonolithicJsonIndex.flush
Hierarchical index	data/datamarts/inverted_index/ <L>/<term>.txt	HierarchicalFolderIndex.flush
Mongo index	database search_engine, collection inverted_index	MongoInvertedIndex.flush
Progress	data/control/downloaded_books.txt, indexed_books.txt	ControlFiles.mark*
Benchmark results	benchmarks/results/ java_<experiment>.csv	CsvResults.write

3. Design decisions

3.1 Structures selected for the final implementation

Every structure required by the assignment is implemented and can be selected in `config.properties` without changing code, so the benchmarks compare them on equal terms. The structures selected in `config.properties`, the most efficient ones according to the results of Section 7, are justified here.

Datalake: book (selected in `config.properties`). It is the most efficient layout in the experiments that the pipeline actually performs:

- *Fastest write:* 1,031 books/s, against 1,006 for **range** and 948 for **time**.
- *Fastest lookup:* the path is computed from the id, $3.3\ \mu\text{s}$ per book against $8.3\ \mu\text{s}$ for **range** and $94.5\ \mu\text{s}$ for **time**, which has to walk its hour folders and gets slower as the ingestion runs for more hours.
- *Fastest detection of new books:* 1.24 ms, against 2.58 and 2.08 ms.
- *Same reliability:* it recovers the 20 interrupted books with no losses or duplicates, like the other two. Its recovery time (23.8 ms) is 4 ms slower than **time**'s, which is negligible.
- *One datalake for everything:* the benchmarks read a **book** datalake (SPEC 10.1), so a single pipeline run serves both to search and to benchmark.

Its drawback is the number of entries in its root folder, one per book (200 here). That is harmless at this scale, but with tens of thousands of books **range** would be the alternative: also a direct lookup, with at most 2,000 files per folder. **time**, the layout suggested by the assignment, offers traceability and folders bounded by the ingestion rate (20 entries), which would pay off if a later stage processed only the most recent folders; in Stage 1 nothing does, so its slower lookup brings no benefit.

Inverted index: monolithic (selected), with mongo as the scalable option. At the scale of this stage (200 books) **monolithic** wins every time-based experiment: building (1.7 s against 11.3 and 14.3 s), querying ($4.5\ \mu\text{s}$ against 48 and $345\ \mu\text{s}$) and adding a book (85 ms against about 1 s), and it takes the least disk. Its weaknesses are structural: each update rewrites the whole JSON and the whole index lives on the heap (100 MB for 200 books), both growing linearly. Beyond roughly 2,000–2,500 real books, or as soon as several processes need the index (Stage 2), **mongo** is the better choice; it is preferred to **hierarchical**, which has the same update cost but wastes 73

times its data size in file-system blocks. Switching is one line of configuration.

Metadata: SQLite with indexes on author and title. The indexes make queries by author or title about 160 times faster with 100,000 rows, for inserts about 14% slower. SQLite needs no server and is more than enough for metadata at this scale.

3.2 Implementation decisions

Interfaces for every storage component.

Callers do not depend on how something is implemented: the `Indexer` receives an `InvertedIndex` and works the same with a JSON file, folders or MongoDB. It also enables test doubles and contract tests run against every implementation.

Factories and a single assembly point.

Only the factories know that “hierarchical” means a `HierarchicalFolderIndex` in the `inverted_index` folder, and only `SearchEngine` connects the pieces. Without them, that mapping would be copied into `Main` and every benchmark. `Main` only parses arguments, connects and prints.

Write to `.tmp`, then rename.

A rename within one file system is atomic: the file under its final name is either complete or absent. A crash during a write leaves only a `.tmp`, which never counts as valid data.

Header before body.

A book exists only when both files are there. A crash between the two leaves the body missing, so the book does not count as stored and is downloaded again.

Mark the control layer after the operation, never before.

Marking first and then crashing would make the control layer claim a book that does not exist, and that book would be lost for good. Marking afterwards means the worst case is repeating an operation, which is safe because every operation is idempotent (metadata upsert, set-based `addDocument`, `$addToSet` in MongoDB, marks without duplicates, overwriting saves).

Compute before writing when indexing.

The `Indexer` parses and tokenizes everything first, so a failure never leaves metadata and index half updated. If power is lost half-way, the book is not marked as indexed and is simply indexed again on restart.

An ASCII-only tokenizer.

Unicode-aware methods such as `toLowerCase()` behave differently across languages. Treating only A–Z, a–z and 0–9 makes the Java, Python and C++ indexes identical, which the reference term counts verify.

An offline book source.

The sample dataset is served through the same `BookSource` interface as Gutenberg, so the offline mode exercises exactly the production code path.

4. Implementation: packages and classes

For each class: what it is, its single responsibility, its important methods, and the reason for its design. The order follows the path of the data.

4.1 Package model: the data that travels

RawBook record

A freshly downloaded and split book, before it is stored: `id`, `header`, `body`. `BookSplitter` produces it and `Datalake.save` consumes it.

Design rationale. A record is immutable and has `equals`, `hashCode` and `toString` for free: ideal for data that is only handed from one piece to the next.

BookLocation record

Where a book was stored: `id`, `headerPath`, `bodyPath`. It holds paths, not text. `save` and `locate` return it.

Design rationale. It separates *where* a book is from *what* it contains: the `Indexer` reads the text only when it needs it.

BookMetadata record

A book's metadata: `id`, `title`, `author`, `language`, `releaseDate` and the two datalake paths. A field missing from the header is `null` (the SPEC says `NULL`).

Design rationale. Storing the paths in SQLite links the metadata datamart to the datalake: from a search result one can reach the text.

4.2 Package crawler: fetching and splitting

BookSource functional interface

“Where the text of a book comes from”: `Optional<String> fetch(int id)`. Empty if the book is not available (HTTP 404, missing file); an I/O failure is an exception.

Design rationale. It makes the whole system testable without network. In unit tests it is a lambda with invented books; the end-to-end test uses one that counts its calls; the offline mode uses `LocalFileSource`.

GutenbergClient class, implements `BookSource`

Downloads `https://www.gutenberg.org/cache/epub/<id>/pg<id>.txt` with Java's `HttpClient`. It only downloads: it neither splits nor stores.

- Constructor (`connectTimeout`, `requestTimeout`): the timeouts come from `AppConfig`.
- `fetch`: HTTP 2xx → the text; any other status or a network error → empty.

Design rationale. Single responsibility: if Gutenberg changes its URL, only this class changes.

LocalFileSource class, implements `BookSource`

The offline counterpart of `GutenbergClient`: reads `<dir>/pg<id>.txt`, named like the last part of the download URL (`sample_dataset/raw/`). It returns the bytes decoded as UTF-8 and untouched, `\r\n` included, so everything downstream runs exactly as online. A missing file is an unavailable book (empty, like a 404); any other read error is an exception.

Design rationale. Adding the offline mode needed no change in any other class: it is one more `BookSource`, chosen by `SearchEngine.openOffline`. The same design exists in the C++ implementation, so both are tested the same way.

BookSplitter class

Splits the raw text into header and body as the SPEC defines, and drops the footer. Normalises `\r\n` to `\n`. If a marker is missing it returns empty and the book is discarded.

- The body starts at the *end of the line* holding the START marker, not right after the pattern, so the title written on that line is not carried into the body.
- The END marker is searched for only after the START marker.

Design rationale. It is a pure function (no I/O): it is tested with strings, and its output is checked byte by byte against `sample_dataset/book/`.

BookDownloader class

Composite component: `fetch` → `split` → `save`. Returns the `BookLocation`, or empty if the book is unavailable or has no markers, in which case nothing is written.

```
return source.fetch(bookId)                // Optional<String>
      .flatMap(text -> splitter.split(bookId, text)) // Optional<RawBook>
      .map(datalake::save);                  // Optional<BookLocation>
```

Design rationale. It does not touch the control files: the caller (`PipelineController`) decides. So it can be reused and tested in isolation.

4.3 Package datalake: storing the books

Datalake interface

The contract of any physical layout: `name()`, `save(RawBook)`, `locate(id)`, `listBookIds()`. The rest of the system only knows this interface.

- `locate` only returns complete books (header and body under their final names).
- `listBookIds` returns sorted ids without repetitions, ignoring `.tmp` files and any other stray file.

Design rationale. A contract test (`DatalakeContractTest`, 36 cases) runs against the three implementations and guarantees that they behave the same.

AbstractFileDatalake abstract class

What the three file-based datalakes share. Subclasses only decide *where* each book goes; this class defines *how* it is written and what counts as a stored book.

- `writeBook`: header first, then body; a book exists only when both are there.
- `isCompleteBook`, `parseCanonicalId` (rejects 0084, -5 and ids of more than 9 digits).
- `writeAtomically`: create the folder, write `x.tmp`, move it to `x`:

```
static void writeAtomically(Path target, String content) throws IOException {
    Files.createDirectories(target.getParent());
    Path tmp = target.resolveSibling(target.getFileName() + TMP_SUFFIX);
    try {
        Files.writeString(tmp, content, StandardCharsets.UTF_8);
        try {
            Files.move(tmp, target, ATOMIC_MOVE, REPLACE_EXISTING);
        } catch (AtomicMoveNotSupportedException e) {
            Files.move(tmp, target, REPLACE_EXISTING); // no atomic move on this file system
        }
    }
}
```

```

    }
} catch (IOException e) {
    Files.deleteIfExists(tmp);
    fails
    throw e;
}
}

```

Design rationale. Template method pattern. The delicate part (atomicity, the definition of a complete book) is written once and cannot diverge between layouts.

BookBasedDatalake class, layout book

One folder per book: <root>/<id>/header.txt and body.txt.

Design rationale. The simplest layout: `locate` computes the path directly. Drawback: the root holds as many entries as there are books (200 in the benchmark; a problem with millions).

RangeBasedDatalake class, layout range

Folders by ranges of 1000 ids: 01000-01999/1342.body.txt.

Design rationale. The path is also computed from the id (direct lookup), and no folder can exceed 2,000 files. `listBookIds` also checks that each file is in its correct range.

TimeBasedDatalake class, layout time

Folders by date and hour of download: 20260930/19/84.body.txt. It receives a `Clock` so tests and benchmarks can simulate time.

- `save` reads the clock once, so header and body land in the same folder even if the hour changes in between.
- `locate` cannot compute the path: it walks the folders from newest to oldest. If an id appears twice, the newest copy wins.
- Patterns `yyyyMMdd` and `HH` (not `YYYY`, the ISO week year, nor `hh`, the 12-hour clock).

Design rationale. It records when each book arrived, which suits incremental ingestion, at the cost of a slow lookup that the benchmark makes visible.

DatalakeStats record

A snapshot of a folder: directories, files, bytes, and the largest number of entries in one directory. Used by the benchmarks.

4.4 Package `datamart.metadata`: metadata in SQLite

MetadataParser class

Extracts `title`, `author`, `release_date` and `language` from the header with the SPEC's regular expressions. It reads no files: it receives the text and the `BookLocation` (for the id and paths).

- MULTILINE regexes: `^` and `$` match the start and end of each line.
- `release_date` drops what is inside brackets: "March 3, 2001 [eBook #10]" → "March 3, 2001".
- A value that is empty after trimming is treated as missing (`null`).

MetadataRepository interface

Metadata store: `save`, `saveAll`, `findById`, `findByAuthor`, `findByTitle`, `count`, `clear`, `close`.

- `save` and `saveAll` are idempotent (saving an id again updates it); `saveAll` is all-or-nothing.
- Searches use exact equality and return results sorted by id.

Design rationale. `close()` is redeclared without `throws Exception`, so the repository can be used in try-with-resources without catching anything.

SqliteMetadataRepository class

JDBC implementation over one SQLite file. It keeps one connection open until `close()` and creates the folder and the schema when opened.

- Upsert: `INSERT ... ON CONFLICT(book_id) DO UPDATE`, so re-indexing a book neither fails nor duplicates it.
- `saveAll`: one transaction with `addBatch/executeBatch` and a single commit; rollback if anything fails.
- Always `PreparedStatement`, never concatenated SQL (no injection, no quoting bugs).

Design rationale. `SQLException` is wrapped in `MetadataRepositoryException`, so callers do not depend on JDBC.

SqliteSchema utility class

The `CREATE TABLE books` and the indexes `idx_books_author` and `idx_books_title`, all with `IF NOT EXISTS` so they run on every open. It also builds the JDBC URL.

MetadataRepositoryException unchecked exception

An error of the metadata store; wraps the original `SQLException`.

4.5 Package datamart.index: the inverted index

An inverted index stores, for each term, the list of books that contain it (its *posting list*). It is “inverted” because a book contains terms, and the index turns that around: from the term to the books. Example: `whale` $\rightarrow$ `[84, 2701]`, `island` $\rightarrow$ `[76, 84, 2701, ...]`.

Tokenizer class

Turns text into terms following the SPEC. `tokenize` returns every token with repetitions; `uniqueTerms` returns each term once, in order of first appearance (what a book contributes); `fromStopwordsFile` loads `shared/stopwords.txt`. Both share one loop:

```
for (int i = 0; i < text.length(); i++) {
    char c = text.charAt(i);
    if (c >= 'A' && c <= 'Z') {
        current.append((char) (c + ('a' - 'A')));    // ASCII uppercase -> lowercase
    } else if ((c >= 'a' && c <= 'z') || (c >= '0' && c <= '9')) {
        current.append(c);
    } else {
        emit(current, sink);                        // separator: closes the token
    }
}
emit(current, sink);                               // the text may end in the middle of a
token
```

`emit` drops tokens shorter than 2 characters and stopwords.

Design rationale. It uses neither `toLowerCase()` nor `Character.isLetter()`: they follow Unicode rules that the C++ implementation cannot reproduce byte for byte. That is why “café” gives “caf” in every language, and why the term counts match exactly across implementations.

InvertedIndex interface

Contract: `name`, `addDocument(id, terms)`, `postings(term)`, `flush`, `clear`, `diskUsageBytes`, `close`.

- `postings` is sorted, without repetitions and unmodifiable; empty if the term does not exist. It sees what was added even before `flush`.
- `addDocument` is idempotent; `clear` also empties the disk.
- After `flush`, closing and reopening keeps everything.

Design rationale. `InvertedIndexContractTest` (33 cases) checks these rules on every implementation.

Implementation	What <code>flush</code> does	Where the index lives
<code>InMemoryInvertedIndex</code>	Nothing (not persistent).	<code>HashMap<String, TreeSet<Integer></code> in memory.
<code>MonolithicJsonIndex</code>	Rewrites the whole JSON (temp file + move).	All in memory (<code>TreeMap</code>); fully loaded when opened.
<code>HierarchicalFolderIndex</code>	Rewrites only the files of the terms that changed.	One file per term; only pending changes in memory.
<code>MongoInvertedIndex</code>	One <code>bulkWrite</code> with an upsert + <code>\$addToSet</code> per term.	In the MongoDB server; only pending changes in memory.

Table 4: The four index implementations.

InMemoryInvertedIndex class, structure memory

A map `term → TreeSet` of ids: `computeIfAbsent(term).add(id)`, and the `TreeSet` keeps ids sorted and unique. It is the “perfect” reference in tests and benchmarks, and exposes `terms()` and `termCount()` to check the reference values.

MonolithicJsonIndex class, structure monolithic

The whole index in a `TreeMap<String, TreeSet<Integer>`, and on disk one JSON file written with Jackson. Opening loads the whole file; `flush` rewrites it entirely if anything changed (*dirty* flag).

- A corrupt JSON throws an exception instead of starting empty: otherwise the next `flush` would overwrite the good index with an empty one.

Design rationale. Very fast queries (everything in memory) and one simple file; but each update costs as much as writing the whole index, and the whole index must fit in RAM.

HierarchicalFolderIndex class, structure hierarchical

One file per term, one id per line: `B/boat.txt`. The folder is the term’s first letter in uppercase; terms starting with a digit go to that digit’s folder.

- `addDocument` accumulates in `pending`; `flush` reads, merges and atomically rewrites only the files of the terms touched.

- `postings` reads only the requested term's file and adds what is pending.
- Validates that the term matches `[a-z0-9]+` (no odd paths, no problems on case-insensitive file systems).

Design rationale. Adding a book only touches its own terms, and memory use is minimal; in exchange there are many thousands of small files (much space wasted in 4 KB blocks) and every query is a disk read.

MongoInvertedIndex class, structure mongo

Documents `{"term": "boat", "postings": [10, 20]}` with a unique index on `term`. It accumulates in memory and `flush` sends everything in one unordered `bulkWrite`.

- `$addToSet` with `$each`: adds only ids not already present (idempotent).
- MongoDB does not guarantee array order: `postings` sorts when reading.
- `diskUsageBytes = storageSize + totalIndexSize`, after an `fsync` (without it, WiredTiger may not have written the data yet).
- 3-second server selection timeout if MongoDB does not answer.

Design rationale. Cheap updates and a server that handles caching, compression and concurrency; the cost is a network round trip for every query term.

Indexer class

The bridge from the datalake to the datamarts for one book: `index(id)`.

1. Read and compute everything without writing anything (header, body, metadata, terms).
2. Save the metadata.
3. `addDocument` and `flush`.

It returns empty, and touches nothing, if the book is not in the datalake.

Design rationale. Computing before writing avoids half-done states. Metadata goes before the index: a book with metadata but no index entry is invisible, while the opposite would return results without a title.

4.6 Package query: the search

SearchService class

AND search: query → terms (same tokenizer) → posting lists → intersection. It never reads books.

- If one term has no postings it returns `[]` without asking for the rest (each request costs a disk read or a round trip in `hierarchical` and `mongo`).
- It sorts the lists from shortest to longest: the result can never be longer than the shortest list.

```
static List<Integer> intersect(List<Integer> a, List<Integer> b) {
    List<Integer> result = new ArrayList<>(Math.min(a.size(), b.size()));
    int i = 0, j = 0;
    while (i < a.size() && j < b.size()) {
        int x = a.get(i), y = b.get(j);
        if (x == y) { result.add(x); i++; j++; } // in both: keep it, advance both
    }
}
```

```

        else if (x < y) i++;           // x cannot be in b
        else j++;                     // y cannot be in a
    }
    return result;
}

```

Design rationale. Two pointers over sorted lists take at most $n + m$ steps, $O(n + m)$. The service works the same with any `InvertedIndex`.

4.7 Package control: progress and recovery

BookIdList utility class

Reads a list of book ids (`shared/book_ids.txt`, or `sample_dataset/book_ids.txt` offline): ids in order, without repetitions; empty lines and `#` lines ignored. A line that is not an id is an error, because the file is written by hand and a typo must not go unnoticed.

ControlFiles class

The two control files as sets: `readyToIndex` = downloaded – indexed.

- Loaded once; each mark is appended to the end of the file, first on disk and then in memory.
- When loading, only complete lines (ending in `\n`) count; a half-written tail is trimmed so the next append cannot form a false id (`"13" + "84\n" = "1384"`).

PipelineController class

Decides what to do next and asks whoever knows how to do it: it does not download, parse or index.

- `step()`: at most one operation. Priority: index what is downloaded; otherwise download the next book of the dataset; otherwise IDLE.
- Marks the control layer only after the operation has succeeded.
- Books that fail are skipped for the rest of the run (`skippedThisRun`, in memory only) and retried after a restart.
- `runUntilIdle(max)`: repeats steps until IDLE or the limit.

Design rationale. Everything needed to continue is in the control files; the process may die at any moment without losing or duplicating anything.

StepResult record

What one step did: `action` (`INDEXED`, `DOWNLOADED`, `NOT_AVAILABLE`, `MISSING_FROM_DATA LAKE`, `DOWNLOAD_FAILED`, `INDEX_FAILED`, `IDLE`), `bookId`, and `detail` (the error message).

4.8 Package config and assembly

AppConfig record

All the configuration, and the only place that builds paths (`datalakeDir()`, `metadataDb()`, `monolithicIndexFile()`, `controlDir()`, `sampleRawDir()`, `sampleBookIdsFile()`...).

- `load()`: defaults `< file < MONGO_URI < -D`.

- Validates in the constructor: an unknown structure, an unknown key or a timeout ≤ 0 fails at start-up, not half-way through a run.
- **withDataDir**: the same configuration with another data folder (benchmarks work apart).

Design rationale. A test walks `src/main/java` and fails if a data path appears outside **AppConfig**.

DatalakeFactory utility class

Maps "book" | "range" | "time" to the matching class: the only new **XxxDatalake** of the application. An unknown name throws an exception that lists the valid options.

InvertedIndexFactory utility class

Maps "monolithic" | "hierarchical" | "mongo" | "memory" to an **InvertedIndex**, with the paths and connection data supplied by **AppConfig**.

SearchEngine class

The complete system assembled from an **AppConfig**: datalake, metadata, index, control, pipeline and search. It is the only place where the pieces are connected.

- **open(config)**: with the real Project Gutenberg (used by **Main**).
- **openOffline(config)**: with **LocalFileSource** over `sample_dataset/raw/` and the sample's 15 ids (used by **Main** with `--offline` and by **SampleDatasetTest**).
- **open(config, source)** and **open(config, source, bookIdsFile)**: with any **BookSource** (the end-to-end test, without network).
- **close()**: closes the index and SQLite (the second even if the first fails).

Design rationale. If the tests assembled the pieces themselves, they would test their own assembly and not the program's. With **SearchEngine**, the tests and **Main** share exactly the same wiring.

Main class

Thin command-line layer: parses `[-config file] command [-offline]`, loads **AppConfig**, asks **SearchEngine** for the system and prints the results. No business logic. Configuration and argument errors → message and exit code 2.

4.9 Package benchmark

BenchmarkRunner class

Repeats a task and measures it: for each repetition it runs **setup()** (not timed) and then **task()** between two **System.nanoTime()** calls. It discards the warm-ups and returns one row per measured run.

Design rationale. **.nanoTime** is a monotonic clock meant for intervals; **currentTimeMillis** is wall time and can jump (NTP adjustments).

BenchmarkRow record

One row of the common CSV. Validates that text fields contain no commas or quotes (so the CSV needs no escaping and Python and C++ read it the same way) and formats numbers with a decimal point and at most 3 decimals.

CsvResults utility class

Writes `benchmarks/results/java_<experiment>.csv` at once, with temp file + move: if a benchmark fails, the previous CSV is left untouched.

BenchmarkBooks utility class

Where benchmark books come from (never the network): an existing datalake (sorted by id, as SPEC 10.1 requires), `synthetic` (16 words, for the datalake), or `syntheticZipf` (30,000 words with a Zipf distribution, for the index; query words at ranks 10, 40, 90... so they have different selectivities).

SimulatedClock class

A clock that advances 6 minutes each time it is read: a reproducible ingestion of 10 books per hour, so the `time` layout spreads books over folders as it would in reality.

DatalakeBenchmark, MetadataBenchmark, IndexBenchmark classes

The three benchmark programs, one `main` each. They are described in Section 7.

5. Testing strategy

357 JUnit 5 tests at four levels. The 12 that need MongoDB are skipped (not failed) when no server is reachable, and pass with one.

Level	What it checks	Examples (number of tests)
Unit	One class in isolation, with doubles for its collaborators.	<code>TokenizerTest</code> (17), <code>MetadataParserTest</code> (14), <code>BookSplitterTest</code> (5), <code>ControlFilesTest</code> (13), <code>SearchServiceTest</code> (14), <code>LocalFileSourceTest</code> (3)
Contract	The same tests against every implementation of an interface.	<code>DatalakeContractTest</code> (36), <code>InvertedIndexContractTest</code> (33)
Integration	Several real pieces together, with real disk and databases.	<code>IndexerTest</code> (9), <code>PipelineControllerTest</code> (11), <code>SqliteMetadataRepositoryTest</code> (15), <code>MongoInvertedIndexTest</code> (10)
System	The whole program, assembled as in <code>Main</code> , without network.	<code>EndToEndTest</code> (6), <code>SampleDatasetTest</code> (1)

Table 5: Test levels.

5.1 The end-to-end test

1. Three invented books in the real Gutenberg format (`\r\n`, markers, licence footer), ids 10, 1500 and 2003, in three different ranges.
2. A fake `BookSource` that counts its calls: the test does not depend on the Internet.
3. `SearchEngine.open(config, fake) → pipeline until IDLE`: 3 downloads + 3 indexings.
4. Metadata: title, author, language, release date without “[eBook #10]”, and the body path stored in SQLite is the datalake’s.

5. Ten AND queries with known answers: several terms, uppercase and punctuation, numbers, only stopwords, and words that appear only in the title, author or footer (they must not be found: only the body is indexed).
6. Everything is closed, the location of each part of the state on disk is checked, and the system is reopened with a source that fails if called. The queries must return identical results and nothing is downloaded again.

It is repeated with `time+monolithic`, `book+hierarchical`, `range+monolithic` and `book+mongo`. A second test simulates the process dying without calling `close()`: the state survives, because it is made persistent at each `flush` and each mark, not when closing.

To check that the test is useful, two faults were introduced on purpose in the assembly: forgetting the stopwords (the query “the of” returned [2003]) and storing the control files in a different folder on each start (“the book 10 was downloaded again after the restart”). The test caught both.

5.2 The sample dataset test

`SampleDatasetTest` runs `SearchEngine.openOffline` on the real 15 books of `sample_dataset/` with the `book` layout and the in-memory index. It checks the 30 step results, compares every stored `header.txt` and `body.txt` byte by byte with `sample_dataset/book/`, counts exactly 30,396 terms and 89,727 postings, and checks the answer to “whale island”. It is the test that ties the Java implementation to the group’s shared reference data.

6. Setup and execution

6.1 Requirements

- JDK 17 or newer (the `pom.xml` compiles for 17; tested with OpenJDK 21) and Maven 3.
- MongoDB only for the `mongo` index or its benchmark. Without a server, its tests are skipped (not failed) and the benchmark leaves it out with a warning. The group’s server is started from the repository root with `docker compose up -d` (MongoDB 8.2 on 127.0.0.1:27017).
- Network only for the online `pipeline` command. Tests, benchmarks and the offline mode do not use it.

6.2 Build and test

All commands are run from `java/stage1/`.

```
mvn package          # compiles and runs the 357 tests (12 skipped without MongoDB)
mvn -q dependency:build-classpath -Dmdep.outputFile=target/classpath.txt
CP="target/classes:$(cat target/classpath.txt)"
```

The `pom.xml` does not build a fat jar, so the program runs from the compiled classes plus the library classpath saved in `$CP`. `mvn exec:java -Dexec.mainClass=es.ulpgc.bigdata.Main -Dexec.args="..."` also works, without the classpath step.

6.3 Quick test without network: the sample dataset

The folder `sample_dataset/` at the repository root holds the first 15 books of the dataset in two forms: `raw/pg<ID>.txt`, the files exactly as Project Gutenberg serves them (the *input* of the pipeline), and `book/<ID>/header.txt` and `body.txt`, the same books already split (the *expected*

output). With the `--offline` option, the pipeline reads the raw files instead of downloading them. Everything after fetching a book is the same code as online.

```
java -cp "$CP" es.ulpgc.bigdata.Main pipeline 30 --offline # 15 downloads + 15 indexings
java -cp "$CP" es.ulpgc.bigdata.Main search whale island
java -cp "$CP" es.ulpgc.bigdata.Main status --offline
```

Expected output (the program's messages are in Spanish):

```
offline: libros leídos de ../Stage_1/sample_dataset/raw
datalake=book index=monolithic
DOWNLOADED 1342
INDEXED 1342
...
DOWNLOADED 76
INDEXED 76

3 libros para "whale island" (index=monolithic)
  76 Adventures of Huckleberry Finn
  84 Frankenstein; or, the modern prometheus
 2701 Moby Dick; Or, The Whale

dataset:      15 (sample_dataset)
descargados:  15
indexados:    15
pendientes:   []
```

Every structure can be tried offline with the usual `-D` options (for example `-Dindex.structure=hierarchical`), and `-Ddata.dir=/tmp/sample` keeps the run apart from `data/`. The raw files are byte-identical to what Gutenberg serves, so a later online **pipeline** finds those 15 books already done and only downloads the other 185.

Automatic check. `SampleDatasetTest`, run by `mvn` package, runs this offline pipeline and verifies that (1) the split of every raw file is byte-identical to `sample_dataset/book/<ID>/`, (2) the index has exactly **30,396 distinct terms and 89,727 postings**, the reference values of the sample, and (3) “whale island” returns books 76, 84 and 2701, the same answer as the C++ implementation.

6.4 Commands

```
java -cp "$CP" es.ulpgc.bigdata.Main [--config file.properties] <command>
```

Command	What it does	Network
<code>pipeline [steps]</code>	Runs up to <code>steps</code> steps (default 10). A step is <i>one</i> download or <i>one</i> indexing, so each book needs two steps: <code>pipeline 400</code> processes the 200 books. It can be interrupted and relaunched: it resumes where it stopped.	yes
<code>pipeline [steps] --offline</code>	The same pipeline over the 15 books of <code>sample_dataset/</code> , read from <code>raw/</code> .	no
<code>search <words...></code>	AND search with the active index; prints id and title (from SQLite).	no
<code>status [--offline]</code>	Dataset size, downloaded, indexed and still pending indexing.	no
<code>config</code>	The effective configuration, after applying the file, the environment and <code>-D</code> .	no

Each line printed by `pipeline` is one step result: `DOWNLOADED`, `INDEXED`, `NOT_AVAILABLE` (no such book, or no `START/END` markers), `DOWNLOAD_FAILED`, `INDEX_FAILED` or `MISSING_FROM_DATA LAKE`. Only the first two are recorded in the control layer. A book that fails is skipped for the rest of the run and retried in the next one. A configuration or argument error stops the program with a clear message and exit code 2.

6.5 Configuration: changing structures without touching code

Each value comes from four sources, and later ones win: (1) the defaults in `AppConfig`, (2) `config.properties` or the file given with `--config`, (3) the `MONGO_URI` environment variable, (4) `-Dkey=value` system properties.

```
data.dir = data
shared.dir = ../../shared
sample.dir = ../../sample_dataset           # used by --offline
benchmarks.dir = benchmarks
datalake.structure = book                   # book | range | time
index.structure = monolithic                # monolithic | hierarchical | mongo | memory
mongo.uri = mongodb://localhost:27017
mongo.database = search_engine
mongo.collection = inverted_index
http.connect.timeout.seconds = 10
http.request.timeout.seconds = 15
```

```
java -cp "$CP" -Ddatalake.structure=book -Dindex.structure=hierarchical es.ulpgc.bigdata.Main
pipeline 400
```

A misspelt value (`index.structure=hierarchcal`), an unknown key or a timeout of 0 stops the program at start-up with a message that lists the valid options.

6.6 Running the benchmarks

The benchmarks never use the network. The 200 real books are downloaded once by the pipeline into a `book` datalake, and that folder is passed to the datalake and index benchmarks:

```
java -cp "$CP" -Ddatalake.structure=book es.ulpgc.bigdata.Main pipeline 400
java -cp "$CP" es.ulpgc.bigdata.benchmark.DatalakeBenchmark data/datalake/book
java -cp "$CP" es.ulpgc.bigdata.benchmark.IndexBenchmark 50,100,200 data/datalake/book
java -cp "$CP" es.ulpgc.bigdata.benchmark.MetadataBenchmark 1000,10000,100000
```

Without the `datalake` argument, `DatalakeBenchmark` and `IndexBenchmark` use synthetic books. `sample_dataset/book` is also a valid `datalake` argument for a quick run. The benchmarks should be run with plain `java` rather than through Maven, because `index_memory` measures the heap of the JVM it runs in. Each run overwrites `benchmarks/results/java_*.csv`; the committed runs are archived in `results/real/` and `results/synthetic/`.

7. Benchmarks and results

7.1 Methodology

- **Warm-ups and measured runs:** 2 discarded repetitions (they warm up the JIT, disk caches and connections) and 5 measured ones. Tables give the **median** of the 5, which resists outliers such as a garbage-collector pause.
- **Setup outside the measurement:** clearing, preparing data and opening connections happen in `setup()`, which is not timed. A test checks this with a deliberately slow setup.
- **Same data for everyone:** each structure receives exactly the same list, in the same order.
- **No network:** real books are read from a `book` datalake already filled by the pipeline; synthetic data is generated with a fixed seed.
- **Growing sizes:** each `N` is the `N` books with the lowest ids, so every `N` is a prefix of the next and all languages measure the same books (SPEC 10.1).
- **Verification:** after measuring, the result is checked (books found, rows inserted, posting lists equal to an in-memory reference). A benchmark that measures something wrong fails instead of writing numbers.
- **Output:** one CSV per experiment in the common SPEC format, comparable with Python and C++.

All results use the common CSV format of the SPEC, so they can be compared directly with the Python and C++ implementations of the group; this report discusses the Java results.

Environment. Linux, 12 threads, 15 GB RAM, NVMe SSD, OpenJDK 21, MongoDB 8.2 in Docker (local). Absolute values depend on the machine; what matters are the ratios and how they change as `N` grows.

7.2 Data: real and synthetic books

The main results use the **real** books: they are what the SPEC requires (the same books in the three languages) and the only data with a genuine vocabulary. The synthetic results of the first version are kept as a reference and compared with the real ones in Section 7.6.

Benchmark	Real data (<code>results/real/</code>)	Synthetic data (<code>results/synthetic/</code>)
Datalake	The 200 books of <code>shared/book_ids.txt</code> , downloaded from Gutenberg (129.0 MB).	200 books of 300 KB with 16 distinct words (61.5 MB).
Index	Prefixes of 50, 100 and 200 of those books. With 200: 129,356 terms and 1,581,064 postings.	Books of 5,000 words drawn with Zipf from 30,000. With 200: 29,716 terms.
Metadata	— (200 rows are too few to see SQLite scale).	Generated rows (SPEC 10.2): 1,000, 10,000 and 100,000.

The 200 ids are the 15 original books plus the most downloaded English books of Gutenberg and classic fiction; all of them were checked to download with both markers. Many are classics with low ids: 110 of the 200 are below 1,000, which shows in the **range** layout. The term and posting counts match the reference values of SPEC 10.1 exactly (58,834 / 78,820 / 129,356 terms for N = 50 / 100 / 200).

7.3 Datalake benchmark

Data: the 200 real books (129.0 MB, the same bytes in the three layouts). The **time** layout uses the **SimulatedClock** (10 books per hour).

Experiment	What exactly is measured	Metrics
datalake_write	Saving the N books into an empty datalake (setup: delete the folder).	elapsed (ms), throughput (books/s)
datalake_lookup	<code>locate()</code> of every id, in a fixed random order (seed 42).	elapsed, per_lookup (μ s)
datalake_incremental	With 90% known and 10% new: <code>listBookIds()</code> minus the known ones.	elapsed, detected
datalake_recovery	1 in 10 books has its body left as <code>.tmp</code> (simulated crash); detecting the missing ones and saving them again.	elapsed, recovered, lost, duplicates
datalake_storage	After saving everything: files, folders, max entries per folder, bytes and allocated bytes.	files, directories, ...

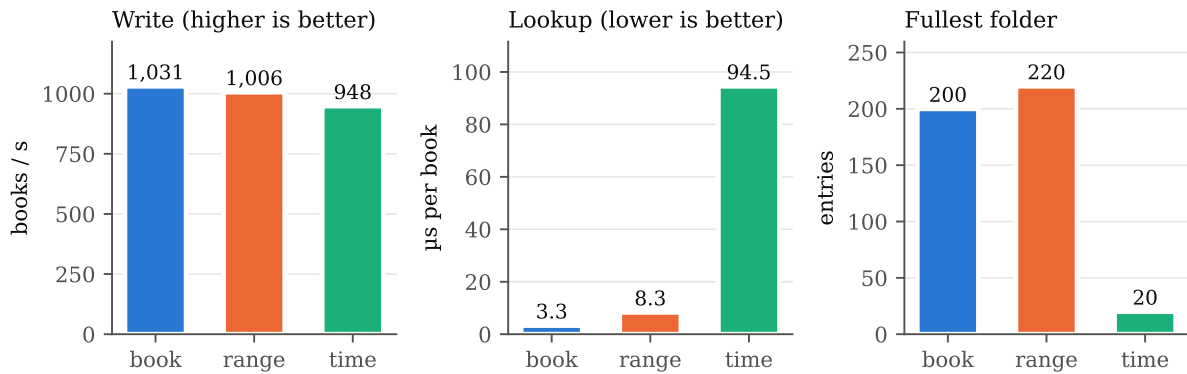


Figure 3: Datalake with the 200 real books (median of 5 runs).

Metric	book	range	time
Total write (ms)	194.0	198.8	210.9
Write (books/s)	1,031	1,006	948
Lookup per book (μ s)	3.3	8.3	94.5
Detect new books (ms)	1.24	2.58	2.08
Recovery (ms)	23.8	25.2	19.8
Books recovered / lost / duplicated	20 / 0 / 0	20 / 0 / 0	20 / 0 / 0
Folders	200	47	21
Max entries in one folder	200	220	20
Data / allocated (MB)	129.0 / 130.9	129.0 / 130.3	129.0 / 130.2

Discussion.

- **Lookup:** `book` (3.3μ s) and `range` (8.3μ s) compute the path from the id: one file check. `time` (94.5μ s, about 29 times `book`) cannot compute it and walks the hour folders from newest to oldest; its cost grows with the number of ingestion hours.
- **Write:** the three take almost the same time, within 9% (194–211 ms, about 1,000 books/s): writing 129 MB and the atomic renames dominate, not the folder layout. With synthetic data the order between them was different (`time` was fastest), a sign that these differences are noise.
- **Incremental and recovery:** all three detect exactly the 20 new books (10%) and recover the 20 damaged ones without losses or duplicates, thanks to the single definition of a “complete book” (a `.tmp` never counts).
- **Storage:** the bytes are identical, but 110 of the 200 ids are between 0 and 999, so `range` puts 110 books (220 files) into 00000–00999: its fullest folder holds more entries than the root of `book`. `range` guarantees at most 2,000 files per folder but does not spread well when ids are concentrated. `time` limits each folder to what arrives in one hour (20 files), whatever the ids.
- **Conclusion:** `book` or `range` when books are looked up by id (the normal case when indexing and showing results); `time` when processing in arrival order matters.

7.4 Metadata benchmark

Data: synthetic rows as SPEC 10.2 defines them: book i has author “Author $[i/10]$ ” and title “Title $[i/2]$ ”. Each author always has 10 books and each title 2, whatever N is: as N grows, only the table grows, not the size of each answer. Header parsing is not timed. Two variants are compared: `sqlite` (with the SPEC’s indexes on `author` and `title`) and `sqlite_no_index` (without them; `book_id` is still the PRIMARY KEY).

This is the only benchmark that stays synthetic, on purpose: it reads ready-made rows, not books, so the real text has no influence, and the difference between an index and a full scan only appears with thousands of rows, not 200.

Experiment	What is measured	Metrics
<code>metadata_insert</code>	<code>saveAll</code> into an empty database, in batches of 1,000 (one transaction each).	elapsed (ms), throughput (rows/s)
<code>metadata_query</code>	1,000 queries of each kind, with existing values (fixed seed); each kind measured separately.	<code>find_by_*</code> (ms) and <code>*_avg</code> (μ s)

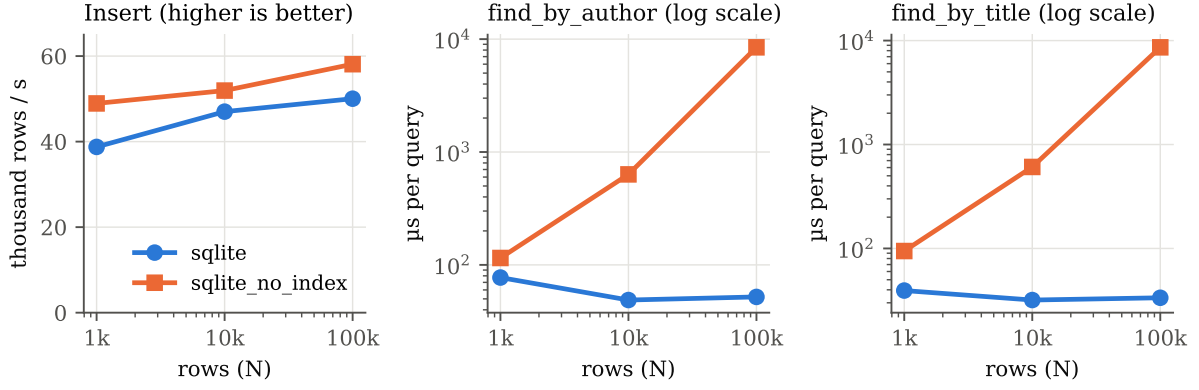


Figure 4: Insert throughput and average query cost as the table grows (logarithmic axes).

Metric (N = 100,000)	sqlite	sqlite_no_index	Ratio
Insert (rows/s)	50,036	58,103	$\times 1.2$
find_by_id (μ s)	24.0	24.4	$\times 1.0$
find_by_author (μ s)	52.0	8,459.6	$\times 162.6$
find_by_title (μ s)	33.5	8,608.9	$\times 256.8$

Discussion.

- **Queries by author or title:** without an index SQLite performs a full scan, so the cost grows linearly with N: from 115.1 μ s with 1,000 rows to 8,459.6 μ s with 100,000. With an index (a B-tree) it is logarithmic and almost flat: 77.2 $\rightarrow$ 52.0 μ s. With 100,000 rows the index is about 160 times faster.
- **find_by_id** is the same in both variants, because both have the PRIMARY KEY (SQLite organises the table by `book_id`). Its 24 μ s is essentially the fixed cost of one JDBC query: the floor for any query in this implementation.
- **Insertion:** maintaining two indexes has a price, since every new row also updates two B-trees, so the variant without indexes inserts faster. This is the classic trade-off: fast reads for slower writes and more disk.
- **Batching:** transactions of 1,000 rows are essential; with autocommit each row would be its own transaction with its own disk sync.

7.5 Inverted-index benchmark

Data: the 200 real books, with the SPEC tokenizer. Sizes: 50, 100 and 200 books (lowest ids first). With 200 books the logical index has 129,356 terms and 1,581,064 postings (term–book pairs), identical in the three backends. Each book contributes 7,905 distinct terms on average.

Fairness rules: all books are tokenized once before measuring (the tokenizer does not pollute the times); before each repetition `freshIndex` clears the backend and checks that it is empty; after measuring, `verify` compares the index with an in-memory one built from the same terms. MongoDB uses its own database, `search_engine_bench`.

Experiment	What exactly is measured	Metrics
<code>index_build</code>	From an empty index to a persisted one: <code>N addDocument</code> with precomputed terms + one <code>flush</code> . Excludes reading, tokenizing, clearing and connecting.	elapsed, throughput
<code>index_query</code>	The index is built and reopened; 100 runs of <code>queries.txt</code> through <code>SearchService</code> (query tokenization included).	elapsed, per_query (μ s)
<code>index_update</code>	Index with $N-k$ books ($k = 10\%$), reopened; adding the k books with one <code>flush</code> per book, as the <code>Indexer</code> does.	elapsed, per_book (ms)
<code>index_memory</code>	Java heap (after requesting GC) with the index built and freshly reopened.	heap_after_build/open
<code>index_disk</code>	After building: logical bytes, files and allocated blocks.	bytes, files, terms, postings

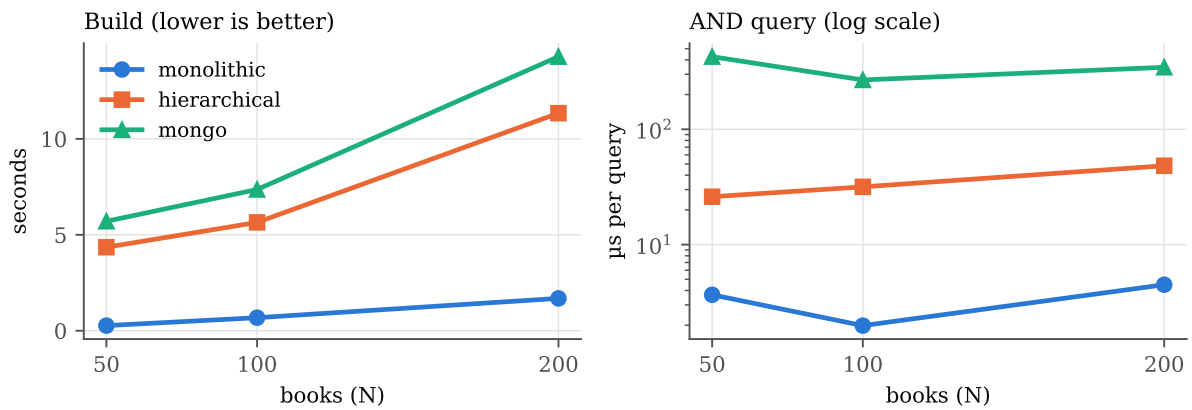
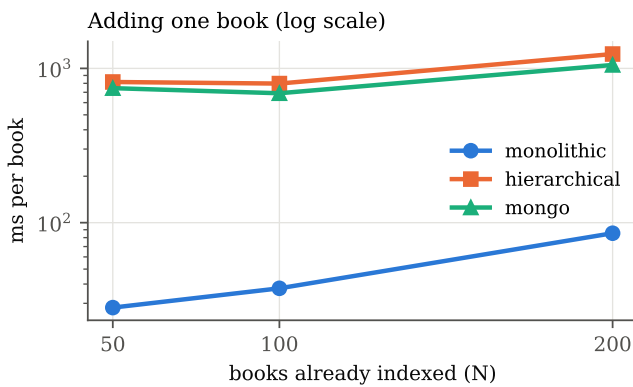


Figure 5: Build time and query cost against the number of books.

Figure 6: Cost of adding one book (log scale). Only `monolithic` grows with the index: its flush rewrites the whole JSON. `hierarchical` and `mongo` only depend on the added book's terms (about 0.1 ms per term in both).

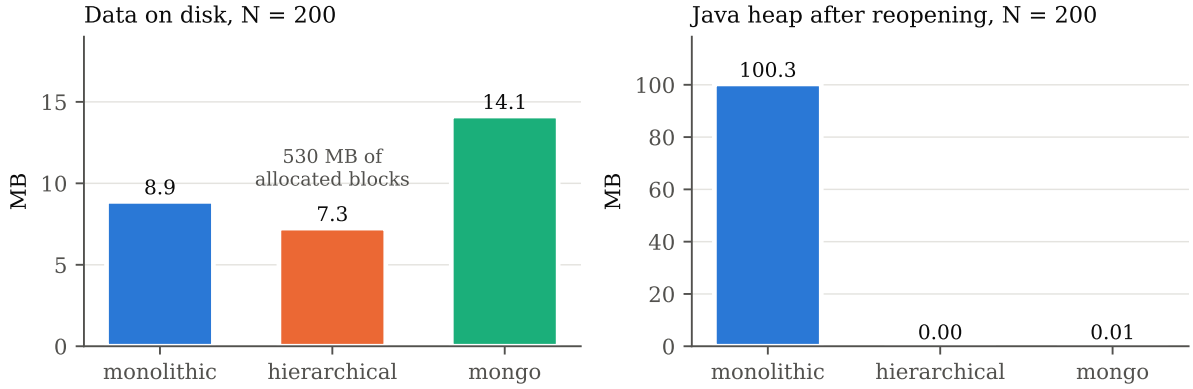


Figure 7: With 200 books: data on disk (left) and Java heap with the index freshly reopened (right).

Metric (median)	N = 50	N = 100	N = 200
Build (s) · monolithic	0.27	0.68	1.69
Build (s) · hierarchical	4.35	5.65	11.34
Build (s) · mongo	5.70	7.36	14.29
Query (μ s) · monolithic	3.7	2.0	4.5
Query (μ s) · hierarchical	26.0	31.7	48.3
Query (μ s) · mongo	426.4	267.9	344.8
Update per book (ms) · monolithic	28.1	37.5	85.4
Update per book (ms) · hierarchical	816.3	797.2	1,239.0
Update per book (ms) · mongo	744.5	690.0	1,052.7
Average terms of the added books	7,592	8,037	10,415
Disk (MB) · monolithic	1.90	3.75	8.89
Disk (MB) · hierarchical	1.15	2.74	7.25
Disk (MB) · mongo	3.98	7.27	14.13
Allocated blocks (MB) · hierarchical	241.1	323.0	530.0
Heap after reopening (MB) · monolithic	18.50	48.29	100.33
Heap after reopening (MB) · hierarchical	0.00	0.00	0.00
Heap after reopening (MB) · mongo	0.01	0.02	0.01
Terms	58,834	78,820	129,356
Postings	360,970	759,087	1,581,064

Discussion.

- **Build:** **monolithic** is by far the fastest (1.69 s for 200 books): everything happens in memory and one file is written at the end. **hierarchical** (11.3 s) writes one file per term (129,356 files, each with its temp file and rename: about 88μ s per file) and **mongo** (14.3 s) sends one upsert per term while the server maintains its unique index. **monolithic**'s throughput falls from 187 to 119 books/s as N grows: the **TreeMap** and the final JSON keep getting larger.
- **Queries:** **monolithic** answers in 4.5μ s because the posting lists are already in memory; **hierarchical** reads one file per term and grows with N because each file holds more ids

($26.0 \rightarrow 48.3 \mu\text{s}$); **mongo** ($268\text{--}426 \mu\text{s}$) is dominated by the round trip to the server for each term, which is why it shows no clear trend with N .

- **Updates, the key result:** **monolithic**'s flush rewrites the entire JSON, so its cost per book grows with the index: $28.1 \rightarrow 37.5 \rightarrow 85.4$ ms, about 10 ms per MB of JSON. **hierarchical** and **mongo** only touch the added book's terms, and their cost per term is almost constant (about 0.1 ms in both). Their rise at $N = 200$ is not caused by the index but by those 20 books being longer (10,415 terms on average against 7,592–8,037).
- **When does monolithic stop paying off?** At these sizes it still wins by a factor of 12, but it is the only curve that grows with the index. At about 10 ms per MB it would reach **mongo**'s 1 s per book when the JSON is around 100 MB; with about 44 KB of JSON per book, that is roughly 2,000–2,500 books. This is an extrapolation, not a measurement.
- **Disk:** the data sizes are of the same order (7.3–14.1 MB). **mongo** takes the most (14.1 MB against 8.9 MB for **monolithic**): it counts its data plus its unique index on **term**. **hierarchical** allocates 530 MB of blocks for 7.3 MB of data ($\times 73$): each of its 129,356 files takes at least one 4 KB block ($129,356 \times 4,096 \text{ B} = 530 \text{ MB}$, exactly what was measured).
- **Memory:** **monolithic** keeps the whole index on the heap: 100 MB with 200 books for an 8.9 MB JSON (about 63 bytes per posting, since each id is an **Integer** inside a **TreeSet**). It grows linearly (about 0.5 MB per book), so the full Gutenberg catalogue (more than 70,000 books) would need tens of GB. **hierarchical** and **mongo** stay near zero after the flush: the data is on disk or in another process.

Limits of the memory measurement. `System.gc()` is a request, not an order, so the value is an estimate with noise of a few KB. For **mongo**, only the client heap (driver, connections, pending changes) is visible: the data lives in the **mongod** process with its own WiredTiger cache, shared by all collections and sized by configuration, so it is not comparable with the Java heap. After building, **hierarchical** and **mongo** leave 0.5–2.1 MB on the heap even though their flush empties the pending map: `HashMap.clear()` keeps the internal table, whose size is a power of two that depends on the number of terms ($131,072 \text{ slots} \times 4 \text{ bytes} = 524,288 \text{ bytes}$ for 50 and 100 books; $524,288 \text{ slots} \times 4 \text{ bytes} = 2,097,152 \text{ bytes}$ for 200, exactly the values measured).

7.6 Real versus synthetic data

The same experiments with $N = 200$, on the synthetic data of the first version and on the 200 real books.

Metric (N = 200)	Synthetic	Real	Real / synthetic
Datalake: bytes stored	61.5 MB	129.0 MB	×2.1
Datalake: lookup · time	143.3 μ s	94.5 μ s	×0.7
Index: terms	29,716	129,356	×4.4
Index: postings	423,261	1,581,064	×3.7
Build · monolithic	0.29 s	1.69 s	×5.9
Build · hierarchical	2.68 s	11.34 s	×4.2
Build · mongo	2.98 s	14.29 s	×4.8
Update per book · monolithic	17.2 ms	85.4 ms	×5.0
Update per book · hierarchical	272.4 ms	1,239.0 ms	×4.5
Update per book · mongo	191.7 ms	1,052.7 ms	×5.5
Disk · monolithic	2.74 MB	8.89 MB	×3.2
Disk · mongo	2.67 MB	14.13 MB	×5.3
Allocated blocks · hierarchical	121.8 MB	530.0 MB	×4.4
Heap after reopening · monolithic	28.71 MB	100.33 MB	×3.5
Query · monolithic	2.6 μ s	4.5 μ s	×1.7
Query · hierarchical	36.6 μ s	48.3 μ s	×1.3
Query · mongo	289.3 μ s	344.8 μ s	×1.2

- **The conclusions hold.** In every index experiment the ranking between backends is the same, and in the datalake **time** is still the only layout with a slow lookup.
- **The magnitudes do not.** Building, updating, disk and memory cost 3 to 6 times more with real books. The synthetic generator limits the vocabulary to 30,000 words and each book to about 2,116 distinct terms, while a real book contributes about 7,905 (proper names, numbers, rare words) and the vocabulary keeps growing: 129,356 terms with 200 books.
- **Queries barely change** (×1.2–1.7): a query only touches the posting lists of its few terms, so it depends much less on the total index size.
- **Only visible with real data:** the uneven spread of ids (**range** with 220 entries in one folder), **mongo** using more disk than **monolithic**, and the update cost depending on which book is added.
- **Lesson:** synthetic data is good for comparing structures with each other; sizing a system (how much disk, RAM or time) needs real data.

7.7 Summary: which structure to choose

If the priority is...	Datalake	Index
Looking books up by id / showing results	book or range	—
Processing in arrival order	time	—
Small folders even when ids are concentrated	time	—
Fastest queries, small index	—	monolithic
Adding books continuously (incremental ingestion)	range	hierarchical or mongo
Index larger than RAM (beyond about 2,000 real books)	—	hierarchical or mongo
Little disk space	any	monolithic
Several processes querying / scaling out	—	mongo

8. Conclusions and future improvements

8.1 Conclusions

The Java implementation delivers the complete Stage 1 data layer: crawler, datalake, metadata and inverted-index datamarts, AND search and a control layer that survives interruptions. All the structures required by the SPEC are implemented behind interfaces and selected by configuration, the 12 benchmark experiments are reproducible and written in the common format, and the 357 tests, including the end-to-end test and the check against the sample dataset, show that the system is correct and recoverable. The sample dataset and the offline mode let anyone verify it in seconds without network.

The benchmarks show that no structure wins everywhere. The **monolithic** index is the best choice at the scale of this stage, but its update cost and memory grow with the index, so a larger collection will need **hierarchical** or, better, **mongo**, which also prepares the system for several processes querying at once. For the datalake, **book** and **range** suit lookups by id, while **time** suits processing in arrival order.

8.2 Limitations and future improvements

- **Control files shared between structures.** Changing the datalake or index structure with existing data leaves books out of the new datalake or index, because the control layer already lists them as done. In practice, delete **data/** (or use another **data.dir**) when switching. A cleaner design, one control folder per combination of structures, would need a change to SPEC section 8 agreed by the group.
- **Small sizes** (≤ 200 real books): enough to see the trends and estimate when **monolithic** stops paying off (about 2,000–2,500 books), but that point is an extrapolation; it should be measured with thousands of books.
- **Prefixes by id.** Each N takes the lowest ids, not a random sample, as the SPEC requires for cross-language comparison. Low ids are older classics and the last books to enter are longer, which is why the update cost rises at $N = 200$.
- **One machine and one run per configuration.** The median of 5 runs resists isolated pauses but not variation between runs (visible in the **mongo** queries).
- **No ranking.** Search returns sorted ids without relevance, since no term frequencies are stored (a SPEC decision). Ranking belongs to the following stages.